

POS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 05 -

IMPLEMENTAÇÃO PRÁTICA COM RABBITMQ

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON.....4

SAIBA MAIS 5

O QUE VOCÊ VIU NESTA AULA?.....10

REFERÊNCIAS 11

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Trabalharemos na integração prática do RabbitMQ com Java, configurando um ambiente completo para envio e recebimento de mensagens. Começaremos com a instalação e inicialização do RabbitMQ, que pode ser feita localmente ou por meio de Docker, e a configuração da interface de gerenciamento para monitoramento em tempo real.

Em seguida, estruturaremos um projeto Java utilizando Maven, adicionando as dependências necessárias e organizando os pacotes. Configuraremos a conexão com o RabbitMQ utilizando a ConnectionFactory, definindo o host e credenciais adequados.

A aula abordará a criação de um Producer em Java para publicar mensagens em uma fila e um Consumer, utilizando classes e métodos específicos. Além disso, veremos como configurar filas com durabilidade e ajustar confirmações manuais (ack).

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Abordaremos a implementação prática do RabbitMQ em conjunto com Java. Começaremos com a configuração do ambiente, incluindo a instalação e inicialização do RabbitMQ, seja localmente ou via Docker, e a ativação da interface de gerenciamento para monitoramento.

No projeto Java, criaremos uma estrutura organizada com pacotes dedicados para Producer, Consumer e Configuração, adicionando as dependências necessárias ao projeto e estabelecendo a conexão com o RabbitMQ por meio da classe ConnectionFactory. Configuraremos o host, as credenciais e uma fila básica para envio e recebimento de mensagens.

Implementaremos um Producer para publicar mensagens em uma fila e um Consumer para processá-las, utilizando callbacks para receber e tratar as mensagens. Também exploraremos configurações avançadas, como filas persistentes, confirmações manuais (ack) e o uso básico de exchanges.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

No RabbitMQ, as mensagens enviadas pelos producers não vão diretamente para as filas. Antes disso, elas passam por um componente chamado Exchange, que desempenha o papel de roteador. As exchanges recebem as mensagens dos producers e, com base em regras de roteamento configuradas, decidem para quais filas essas mensagens serão enviadas.

Essa estrutura permite uma organização mais flexível e eficiente do fluxo de mensagens. Por exemplo, diferentes tipos de mensagens podem ser encaminhados para filas específicas, garantindo que cada consumidor receba apenas os dados relevantes para sua tarefa. Além disso, as exchanges possibilitam padrões como envio para múltiplas filas ou filtragem de mensagens, adaptando-se às necessidades de sistemas complexos.

Essa etapa intermediária entre producers e filas é fundamental para a arquitetura do RabbitMQ, proporcionando maior controle e escalabilidade na comunicação entre sistemas.

No RabbitMQ, existem alguns tipos de exchanges.Umas delas é a Direct Exchange, que é uma exchange que roteia mensagens para filas específicas com base em uma chave de roteamento, conhecida como routing key. Essa chave é definida pelo producer no momento em que a mensagem é enviada e deve corresponder exatamente à chave associada a uma fila para que o roteamento ocorra com sucesso.

Por exemplo, em um sistema de processamento de transações, o producer pode enviar uma mensagem com a routing key "pagamento". A Direct Exchange verifica essa chave e a encaminha para a fila correspondente, como a fila de pagamentos. Da mesma forma, mensagens com routing keys como "faturamento" ou "logística" seriam roteadas para filas distintas, destinadas a esses processos específicos.

Esse tipo de exchange é ideal para cenários onde o roteamento é direto e bem definido, proporcionando um fluxo organizado e preciso para o processamento de mensagens em sistemas distribuídos.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Temos também a Fanout Exchange, um tipo de exchange que envia mensagens para todas as filas conectadas a ela, independentemente de qualquer chave de roteamento. Esse modelo garante que todas as filas vinculadas à exchange recebam a mesma mensagem, tornando-a ideal para cenários em que a mensagem precisa ser distribuída amplamente.

Por exemplo, em um sistema de notificações, uma mensagem enviada por um producer para a Fanout Exchange pode ser encaminhada simultaneamente para filas responsáveis por enviar e-mails, mensagens SMS e notificações push. Assim, múltiplos sistemas podem processar a mesma mensagem de forma independente, garantindo que a notificação alcance diferentes canais. Esse tipo de exchange é amplamente utilizado em casos onde a mensagem deve ser replicada para diversos consumidores, promovendo simplicidade e eficiência na distribuição em larga escala.

A terceira exchange é a Topic Exchange,que roteia mensagens para filas com base em padrões de routing key. Esse tipo de exchange oferece maior flexibilidade ao permitir que as filas usem padrões de filtragem com curingas, como ***** (que corresponde a um único termo) e # (que corresponde a zero ou mais termos), para definir quais mensagens devem ser recebidas.

Por exemplo, em um sistema de logs, uma mensagem com a routing key app.error.log pode ser roteada para uma fila configurada para receber todas as mensagens que terminam com .log. Outra fila, configurada com o padrão app.*.log, receberia mensagens relacionadas a qualquer tipo de log gerado pela aplicação. Da mesma forma, uma fila com o padrão *.error.* capturaria mensagens de erros em diferentes componentes do sistema.

A Topic Exchange é especialmente útil em cenários onde as mensagens precisam ser categorizadas dinamicamente e entregues a consumidores que atendem a critérios específicos, promovendo um roteamento avançado e adaptável em sistemas complexos.

A quarta exchange é a Headers Exchange, que roteia mensagens com base nos cabeçalhos (headers) definidos nas mensagens, em vez de utilizar uma routing key. Esse tipo de exchange permite que as mensagens sejam direcionadas para filas específicas conforme os valores de seus cabeçalhos correspondam a condições predefinidas.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Por exemplo, uma mensagem com o cabeçalho **tipo=relatório** pode ser roteada para uma fila configurada para processar relatórios, enquanto outra com o cabeçalho **prioridade=alta** pode ser direcionada para uma fila que trata mensagens prioritárias. Essa abordagem é útil quando os cabeçalhos da mensagem carregam informações mais detalhadas ou específicas do que uma routing key.

Embora menos comum que outros tipos de exchanges, como Direct ou Topic, a Headers Exchange é uma opção poderosa para cenários que exigem roteamento baseado em atributos complexos das mensagens.

Um conceito muito importante que trabalha com as exchanges é a Binding Key, que define a relação entre uma fila e uma exchange, especificando como as mensagens devem ser roteadas da exchange para a fila. Essa chave funciona como uma regra que conecta a fila à exchange, permitindo que somente as mensagens que atendam a critérios específicos sejam entregues a essa fila.

Por exemplo, em uma Direct Exchange, uma fila pode ser vinculada com a binding key **"pagamento"**, de forma que apenas mensagens com essa routing key sejam enviadas para ela. Em uma Topic Exchange, uma fila com a binding key ***.erro.*** receberá mensagens cujas routing keys correspondam a esse padrão, como **"sistema.erro.log"**.

As binding keys são essenciais para configurar o comportamento de roteamento no RabbitMQ, permitindo que diferentes filas recebam apenas as mensagens relevantes para suas finalidades.

Parecida com as binding Keys, temos as Routing Keys. Elas são chaves definidas pelo producer no momento em que a mensagem é enviada para uma exchange. Essas chaves são usadas para determinar para quais filas a mensagem será roteada, de acordo com as regras configuradas na exchange.

Em uma Direct Exchange, a routing key precisa corresponder exatamente à binding key de uma fila para que a mensagem seja enviada para ela. Por exemplo, uma mensagem com a routing key "pagamento" será entregue apenas à fila vinculada com a mesma binding key.

Já em uma Topic Exchange, a routing key é comparada com padrões mais flexíveis definidos pelas binding keys. Por exemplo, uma mensagem com routing key

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

"sistema.erro.log" será roteada para filas configuradas com padrões como ***.erro.*** ou **sistema.#**.

Para nos aprofundarmos no RabbitMQ, precisamos abordar também os conceitos de **durabilidade e persistência** que são características importantes para garantir que filas e mensagens sobrevivam a falhas ou reinicializações do broker.

A durabilidade é configurada no momento em que uma fila é criada, tornando-a persistente no disco. Isso significa que, mesmo que o servidor RabbitMQ seja reiniciado, as filas permanecerão intactas e disponíveis para uso.

Já a persistência das mensagens é configurada pelo producer no momento do envio, marcando as mensagens como "persistentes". Quando uma mensagem persistente é enviada para uma fila durável, ela será armazenada no disco pelo RabbitMQ. Isso garante que, em caso de falha no sistema, as mensagens armazenadas não sejam perdidas.

Por exemplo, em um sistema de processamento de pedidos, configurar tanto as filas quanto as mensagens como persistentes assegura que nenhum pedido seja perdido, mesmo em situações de falha no servidor. Essa combinação de durabilidade e persistência é essencial para aplicações críticas, onde a confiabilidade e a integridade dos dados são prioridades.

No RabbitMQ, temos também o Ack (acknowledgment) e a rejeição de mensagens, que são mecanismos usados pelos consumidores para confirmar ou rejeitar o processamento de mensagens recebidas.

Quando uma mensagem é entregue a um consumidor, o RabbitMQ espera por uma confirmação (Ack) indicando que a mensagem foi processada com sucesso. Essa confirmação garante que o RabbitMQ possa remover a mensagem da fila. Se o consumidor falhar em processar a mensagem ou optar por rejeitá-la, ele pode explicitamente enviar uma rejeição, sinalizando que a mensagem não foi concluída.

Uma mensagem rejeitada pode ser reencaminhada para a fila ou descartada, dependendo da configuração definida no sistema. Por exemplo, se uma fila está configurada com um mecanismo de Dead Letter Queue (DLQ), mensagens rejeitadas podem ser enviadas para essa fila para análise posterior.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Dead Letter Queue (DLQ) é uma fila especial usada para armazenar mensagens que não puderam ser processadas com sucesso. Quando uma mensagem é rejeitada por um consumidor, expira ou excede o número máximo de tentativas de redirecionamento, ela pode ser enviada para a DLQ.

Essa configuração permite que mensagens problemáticas sejam separadas das filas principais, evitando bloqueios ou falhas no fluxo normal de processamento. As mensagens na DLQ podem ser analisadas posteriormente para identificar problemas, como dados inválidos ou erros de processamento, ajudando a melhorar a confiabilidade e a eficiência do sistema.

Por exemplo, em um sistema de processamento de pedidos, mensagens relacionadas a pedidos inválidos ou com informações incompletas podem ser enviadas para a DLQ, permitindo que sejam investigadas e corrigidas sem impactar o restante do fluxo.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Vimos como configurar e integrar o RabbitMQ com Java, criando um ambiente funcional para envio e recebimento de mensagens. Iniciamos com a instalação e inicialização do RabbitMQ, utilizando configurações via Docker, e exploramos a interface de gerenciamento para monitoramento das filas e mensagens.

No projeto Java, estruturamos pacotes organizados, configuramos a conexão com o RabbitMQ utilizando a ConnectionFactory, e criamos filas básicas para gerenciar mensagens. Implementamos um Producer para enviar mensagens para uma fila e um Consumer para processá-las, com callbacks para lidar com mensagens recebidas.

Além disso, abordamos configurações avançadas, como durabilidade de filas, confirmações manuais (ack) e introduzimos o uso de exchanges como Direct para roteamento de mensagens.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

APACHE KAFKA. Apache Kafka Documentation. 2024. Disponível em: https://kafka.apache.org/. Acesso em: 21 jan. 2024.

AWS. Benefícios do Message Queue. 2024. Disponível em: https://aws.amazon.com/pt/message-queue/benefits/. Acesso em: 21 jan. 2024.

RED HAT. O que é Apache Kafka? 2024. Disponível em: https://www.redhat.com/pt-br/topics/integration/what-is-apache-kafka. Acesso em: 21 jan. 2024.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Mensageria. Message Broker. Fila de Mensagens. Pub/Sub. Producers. Consumers. Queues. Exchanges. Roteamento de Mensagens. AMQP. RabbitMQ. Apache Kafka. AWS SQS. Escalabilidade. Baixo Acoplamento. Persistência de Mensagens. Garantia de Entrega. Resiliência. Comunicação Assíncrona. Sistemas Distribuídos. Eventos. JSON. XML. Confiabilidade. Cloud-Native. Processamento em Lote. Boas Práticas. Integração de Sistemas. Protocolos de Mensageria. Paginação de Mensagens. Alto Desempenho.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com